MEL G641 CAD IC Design

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
(Rajasthan)

(April, 2026)

Project Report on

CORDIC Algorithm implementation for Sine and Cosine functions

Submitted By

Name	Roll Number
Anmol Shree	2025H1230133P
Gaurav Kumar	2025H1230128P
Krishna Jindal	2025H1230127P
Nishant Sahu	2025H1230134P
Vignesh Kenche	2025H1230131P

Submitted To:

Dr. Abhijit Asati

Introduction:

Modern digital systems frequently require the computation of trigonometric, hyperbolic, and other transcendental functions. In traditional software implementations these are realised through polynomial approximations such as Taylor or Chebyshev series expansions, which converge accurately but demand many multiply-accumulate operations. In hardware, dedicated floating-point units achieve high precision yet consume significant silicon area and power. For resource-constrained platforms such as FPGAs, ASICs without hardware multipliers, or embedded microcontrollers, neither approach is ideal.

The CORDIC algorithm — Coordinate Rotation Digital Computer — offers a compelling alternative. First proposed by Jack E. Volder in 1959 for real-time airborne navigation, CORDIC computes a broad family of mathematical functions using nothing more than additions, subtractions, bit-shifts, and a small pre-computed lookup table. This hardware profile maps with near-perfect efficiency onto FPGA logic fabric, where shift operations are free and adders are cheap.

This report presents a complete analysis of a pipelined CORDIC core implemented in Verilog HDL. The design computes sine and cosine across the full 0° to 360° range, accepting a 32-bit fixed-point angle each clock cycle and producing 16-bit Q1.15 results after a latency of 16 cycles. Sections cover the mathematical derivation from first principles, fixed-point number formats, quadrant extension logic, pipeline architecture, and key implementation decisions in the Verilog source.

Motivation for Hardware Implementation

Three properties make CORDIC particularly attractive:

- **No multipliers required:** The inner loop uses only adders and bit-shifts. On an FPGA this means the core can be synthesised entirely from LUT and flip-flop resources, leaving DSP slices free for other functions.
- **Pipeline able architecture:** Each CORDIC iteration is an independent register-to-register stage. Inserting flip-flops between stages yields a fully pipelined implementation with throughput of one result per clock cycle.
- **Scalable precision:** Accuracy scales directly with the number of iterations. Adding one stage gains approximately one additional bit of precision, allowing the designer to trade area for accuracy continuously.

Scope of This Implementation

This implementation targets the following specifications:

- **Input:** 32-bit signed fixed-point angle in Q2.30 format covering 0–360°
- **Output:** 16-bit signed fixed-point sine and cosine in Q1.15 format
- **Architecture:** Fully pipelined systolic array (new input accepted every clock cycle)
- **Iterations:** 15 CORDIC stages (width−1), yielding approximately 15-bit accuracy
- **Latency:** 16 clock cycles from input to valid output
- **Platform:** Synthesisable Verilog HDL, targeting any FPGA family

Coordinate Rotation Fundamentals

The foundation of CORDIC is the rotation of a 2D vector by an arbitrary angle φ. Given an initial vector (x_0, y_0), the components of the rotated vector (x_n, y_n) are given by the standard rotation matrix equations:

$x_n = x_0 \cos\varphi - y_0 \sin\varphi$

$y_n = y_0 \cos\varphi + x_0 \sin\varphi$

Factoring out cosφ from both equations, and recognising that sinφ/cosφ = tanφ:

$x_n = \cos\varphi \cdot (x_0 - y_0 \tan\varphi)$

$y_n = \cos\varphi \cdot (y_0 + x_0 \tan\varphi)$

The key insight of Volder's algorithm is to restrict the allowed rotation angles such that $\tan\varphi = \pm 2^{-i}$ for integer i. This turns the multiplication by tanφ into a simple arithmetic right-shift by i positions, eliminating all multiplications from the iterative loop. The direction d_i of each micro-rotation is chosen to drive the residual angle accumulator z toward zero:

$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i}$

$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i}$

$z_{i+1} = z_i - d_i \cdot \arctan(2^{-i})$

where $d_i = +1$ if $z_i \geq 0$, else −1. In the Verilog implementation this is captured by z_sign = z[i][31] (the MSB, which is the sign bit in two's complement). When z_sign = 1 (z is negative), the rotation was overshot and must be partially reversed.

The CORDIC Gain Factor K

Because the permitted rotation angles are arctan(2^{-i}) rather than exactly 2^{-i}, each iteration scales the vector magnitude by a factor of cos(arctan(2^{-i})) = $1/\sqrt{(1+2^{-2i})}$. After N iterations the cumulative scale factor applied to the vector is:

$$A_n = \prod\sqrt{(1 + 2^{-2i})} \quad \text{for } i = 0 \text{ to } N-1 \quad \approx 1.647$$

To obtain unit-amplitude sine and cosine outputs the starting x value must be pre-multiplied by the reciprocal gain K = $1/A_n$. This is the classic approach of absorbing the gain correction into the initial conditions, avoiding any post-multiplier. For 15 iterations:

$$K = 1 / 1.647 \approx 0.607252935$$

Arctangent Lookup Table Generation

Each CORDIC iteration i subtracts a pre-computed constant arctan(2^{-i}) from the angle accumulator z. These constants are stored in the atan_table array. The values are normalised to the Q2.30 fixed-point format, where a full 360° circle corresponds to 2^{32} counts:

$$atan\_table[i] = round(arctan(2^{-i}) / (2\pi) \times 2^{32})$$

Derivation of index 0: arctan(2^0) = arctan(1) = 45°. As a fraction of 360°: 45/360 = 1/8. So atan_table[0] = $2^{32}/8 = 2^{29}$ = 0x20000000 ✓. The complete table:

Index i	Angle (degrees)	Hex Value
0	45.000°	0x20000000
1	26.565°	0x12E4051D
2	14.036°	0x09FB385B
3	7.125°	0x051111D4
4	3.576°	0x028B0D43
5	1.790°	0x0145D7E1
6	0.895°	0x00A2F61E
7	0.448°	0x00517C55
8	0.224°	0x0028BE53
9	0.112°	0x00145F2E
10	0.056°	0x000A2F98
11	0.028°	0x000517CC
12	0.014°	0x00028BE6
13	0.007°	0x000145F3
14	0.003°	0x0000A2F9
15	0.002°	0x0000517C

Quadrant Extension to Full 360°

The basic CORDIC algorithm converges only for input angles in the range [−π/2, +π/2]. Full-circle coverage is achieved by exploiting standard trigonometric symmetry identities that map any angle into the convergence region before the iterative stages begin

The 32-bit angle input uses a Q2.30 format. The two MSBs (`angle[31:30]`) directly encode the quadrant, while bits [29:0] encode the fractional displacement within that quadrant. Because a full 360° circle maps to 2^{32} counts, π/2 = 2^{30} = 0x40000000 exactly — a power of two. This alignment is deliberate: it makes quadrant detection a two-bit compare and π/2 offset a zero-cost bit substitution

angle[31:30]	Quadrant	Transformation Applied
2'b00	1st (0° to 90°)	No change — direct computation
2'b11	4th (270° to 360°)	No change — direct computation
2'b01	2nd (90° to 180°)	x_0 = −y_start, y_0 = x_start, z = {2'b00, angle[29:0]}
2'b10	3rd (180° to 270°)	x_0 = y_start, y_0 = −x_start, z = {2'b11, angle[29:0]}

Bit-Level Offset Arithmetic

Because π/2 in Q2.30 is exactly 0x40000000 (bits 31:30 = 01, all lower bits zero), subtracting or adding π/2 reduces to replacing the top two bits of the angle while leaving bits [29:0] unchanged. In the 2nd quadrant case, replacing `01` with `00` is equivalent to subtracting π/2. In the 3rd quadrant, replacing `10` with `11` (which in two's complement is −1/2 circle + angle_frac, effectively adding π/2) maps the angle to the equivalent negative-quadrant value. This requires no adder, carries no delay, and consumes no LUT resources.

1.Code:

```
//CORDIC implementation for sine and cosine for Final Project

`timescale 1ns/1ps
module CORDIC(clk, rst, valid_in, angle, cosine, sine, valid_out);

 parameter width = 16;
 parameter N = 32;

 // Inputs
 input clk;
 input rst;
 input valid_in;
 input signed [N-1:0] angle;

 // Outputs
 output signed  [width-1:0] sine, cosine;
 output valid_out;

 wire signed [width-1:0] x_start,y_start;

 assign x_start = 16'h4DB0; // 0.607252935 in Q1.15 format, pre-scaled by K factor to ensure output is in range
 assign y_start = 16'h0000; // start with y=0 for sine/cosine calculation

 // Generate table of atan values
 wire signed [N-1:0] atan_table [0:width-1];

 assign atan_table[00] = 32'h20000000; // 45.000 degrees -> atan(2^0)
 assign atan_table[01] = 32'h12E4051D; // 26.565 degrees -> atan(2^-1)
 assign atan_table[02] = 32'h09FB385B;
 assign atan_table[03] = 32'h051111D4;
 assign atan_table[04] = 32'h028B0D43;
 assign atan_table[05] = 32'h0145D7E1;
 assign atan_table[06] = 32'h00A2F61E;
 assign atan_table[07] = 32'h00517C55;
 assign atan_table[08] = 32'h0028BE53;
 assign atan_table[09] = 32'h00145F2E;
 assign atan_table[10] = 32'h000A2F98;
 assign atan_table[11] = 32'h000517CC;
 assign atan_table[12] = 32'h00028BE6;
 assign atan_table[13] = 32'h000145F3;
 assign atan_table[14] = 32'h0000A2F9;
 assign atan_table[15] = 32'h0000517C;



 reg signed [width:0] x [0:width-1];
 reg signed [width:0] y [0:width-1];
 reg signed    [N-1:0] z [0:width-1];

 reg [width-1:0] valid_pipe;

 // make sure rotation angle is in -pi/2 to pi/2 range
 wire [1:0] quadrant;
 assign quadrant = angle[N-1:N-2];

 always @(posedge clk)
```

```
begin // make sure the rotation angle is in the -pi/2 to pi/2 range
  if(rst)
  begin
     x[0] <= 0;
     y[0] <= 0;
     z[0] <= 0;
  end
  else
  begin
     case(quadrant)
     // 1st and 4th quadrants - no changes needed
     2'b00,
     2'b11: // no changes needed for these quadrants
     begin
        x[0] <= x_start;
        y[0] <= y_start;
        z[0] <= angle;
     end
     // 2nd quadrant - rotate by -90 degrees
     2'b01:
     begin
        x[0] <= -y_start;
        y[0] <= x_start;
        // 90 degree = {{2'b01}, {30'b0}};
        // so set angle[31:30] from 01 to 00 and keep the rest of the bits the same to effectively subtract 90 degrees from the angle
        z[0] <= {2'b00,angle[29:0]}; // subtract pi/2 for angle in this quadrant
     end
     // 3rd quadrant - rotate by +90 degrees
     2'b10:
     begin
        x[0] <= y_start;
        y[0] <= -x_start;
        // 90 degree = {{2'b01}, {30'b0}};
        // so set angle[31:30] from 10 to 11 and keep the rest of the bits the same to effectively add 90 degrees from the angle
        z[0] <= {2'b11,angle[29:0]}; // add pi/2 to angles in this quadrant
     end
     // Catch X and Z states!
     default: begin
           x[0] <= 0;
           y[0] <= 0;
           z[0] <= 0;
        end
     endcase
  end
end


// run through iterations
// CORDIC PIPELINE
genvar i;

generate
for (i=0; i < (width-1); i=i+1)begin: stages
  wire z_sign;
  wire signed [width:0] x_shr, y_shr;

  assign x_shr = x[i] >>> i; // signed shift right: x * 2^-i
  assign y_shr = y[i] >>> i; // signed shift right: y * 2^-i

  assign z_sign = z[i][31]; //the sign of the current rotation angle
```

```
  always @(posedge clk)
  begin
    if(rst)
    begin
      x[i+1] <= 0;
      y[i+1] <= 0;
      z[i+1] <= 0;
    end
    else
    begin
      // add/subtract shifted data based on the sign of the current angle
      // z_sign = 1(means Z is negative), so we need to rotate in the reverse direction and adding the atan value to z(overshot)
      // z_sign = 0(means Z is positive), so we need to rotate in the forward direction and subtracting the atan value from z
      // add/subtract shifted data
      x[i+1] <= z_sign ? x[i] + y_shr : x[i] - y_shr;
      y[i+1] <= z_sign ? y[i] - x_shr : y[i] + x_shr;
      z[i+1] <= z_sign ? z[i] + atan_table[i] : z[i] - atan_table[i];
    end
  end
 end
 endgenerate

 // VALID PIPELINE
  always @(posedge clk)begin
    if(rst)
      valid_pipe <= 0;
    else
      valid_pipe <= {valid_pipe[width-2:0], valid_in}; // shift in the valid signal through the pipeline
      // valid_pipe[0] gets valid_in,
      // valid_pipe[width-1:1] gets valid_pipe[width-2:0], effectively shifting the valid signal down the pipeline each clock cycle until
it reaches the end when the output is ready
  end

  assign valid_out = valid_pipe[width-1]; // output valid signal when the data is ready at the end of the pipeline

 // assign output
 assign cosine = x[width-1];
 assign sine = y[width-1];

endmodule
```

2.Testbench:

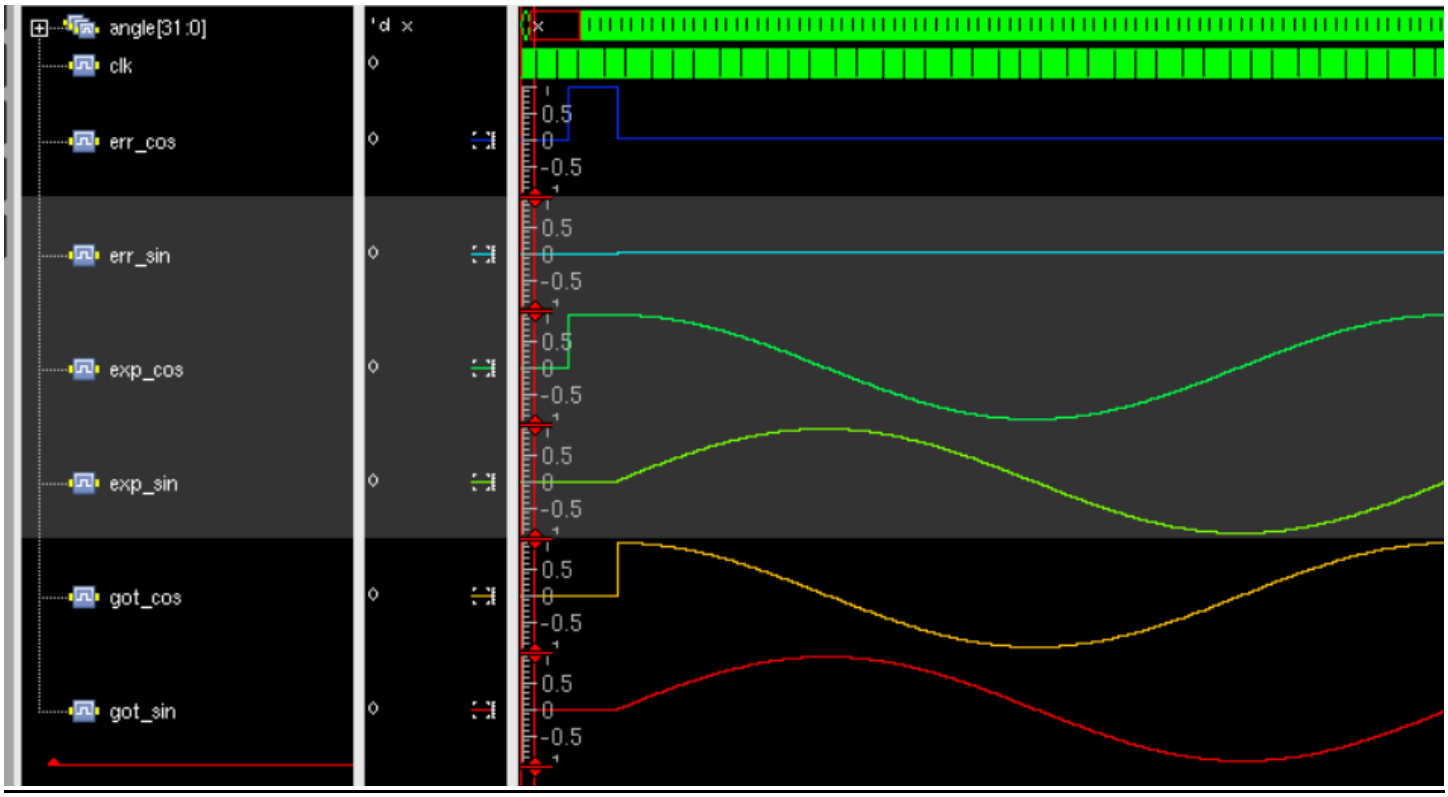
```
`timescale 1ns/1ps

module cordic_tb;

 // ------------------------------------------------------------
 // Parameters
 // ------------------------------------------------------------
 parameter width = 16;
 parameter N = 32;
```

```
// ------------------------------------------------------------
// DUT Signals
// ------------------------------------------------------------
reg clk;
reg rst;
reg valid_in;
reg signed [N-1:0] angle;

wire signed [width-1:0] cosine;
wire signed [width-1:0] sine;
wire valid_out;

// ------------------------------------------------------------
// Instantiate DUT
// ------------------------------------------------------------
CORDIC dut (
  .clk(clk),
  .rst(rst),
  .valid_in(valid_in),
  .angle(angle),
  .cosine(cosine),
  .sine(sine),
  .valid_out(valid_out)
);

// ------------------------------------------------------------
// Clock Generation (100 MHz)
// ------------------------------------------------------------
always #5 clk = ~clk;

// ------------------------------------------------------------
// Testbench Variables
// ------------------------------------------------------------
integer i;
real deg;
real exp_cos, exp_sin, got_cos, got_sin, err_cos, err_sin;

real PI = 3.14159265358979323846;
real SCALE = 32767.0;

// Queue to track which degree corresponds to the current output
// Expanded to handle thousands of random tests
real deg_queue [0:5000];
integer push_idx = 0;
integer pop_idx  = 0;

function real rabs(input real v);
  begin
    rabs = (v >= 0.0) ? v : -v;
  end
endfunction

// ------------------------------------------------------------
// Main Stimulus Block (Feeding the Pipeline)
// ------------------------------------------------------------
initial begin
  $dumpfile("cordic_tb.vcd");
  $dumpvars(0, cordic_tb);

  // 1. Initial State & Power-on Reset
```

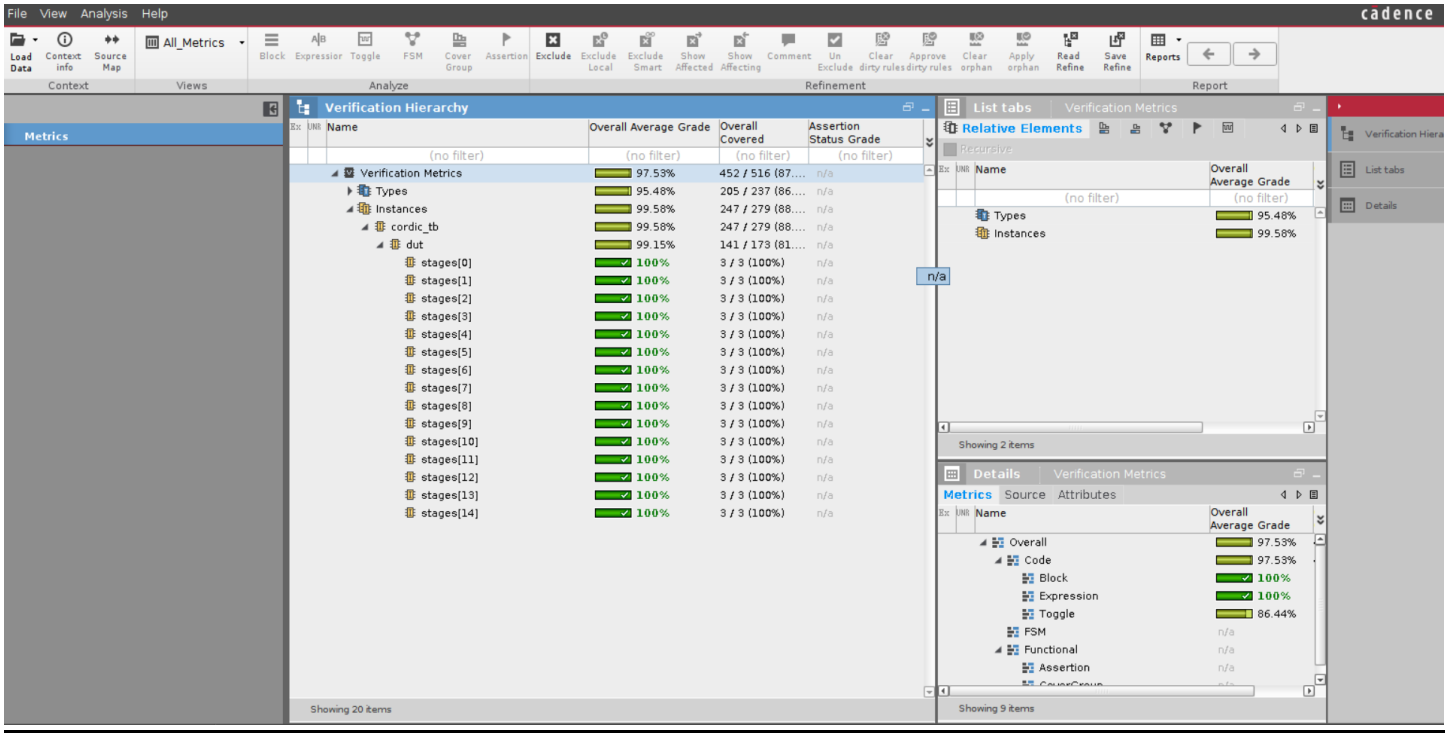
```
clk = 0;
rst = 1;
valid_in = 0;
angle = 0;
repeat(3) @(negedge clk);
rst = 0;

// ========================================================
// TEST PHASE 0: The "Garbage" Injection
// Goal: Cover the 'default' case by forcing Unknowns (X)
// ========================================================
$display("Starting Phase 0: Injecting Unknowns (X)...");
@(negedge clk);
valid_in = 1;
angle = 32'hXXXX_XXXX; // Force the quadrant wire to become 2'bXX
@(negedge clk);
valid_in = 0;
// Wait for the pipeline to clear the garbage
repeat(20) @(posedge clk);


// ========================================================
// TEST PHASE 1: Sequential Sweep (0 to 360)
// Goal: Statement and Block Coverage
// ========================================================
$display("Starting Phase 1: 0 to 360 Sweep...");
for (i = 0; i <= 360; i = i + 1) begin
  @(negedge clk);
  deg = i * 1.0;
  angle = $rtoi((deg / 360.0) * 4294967296.0);
  valid_in = 1;
  deg_queue[push_idx] = deg;
  push_idx = push_idx + 1;
end

// ========================================================
// TEST PHASE 2: Pipeline Bubbles & Randomized Data
// Goal: Toggle Coverage & Control Path Verification
// ========================================================
$display("Starting Phase 2: Random Angles & Pipeline Bubbles...");
for (i = 0; i < 2000; i = i + 1) begin
  @(negedge clk);

  // Generate a completely random 32-bit angle
  angle = {$random, $random};

  // Back-calculate the degree for the monitor checking queue
  // Modulo arithmetic to keep it in a readable 0-360 range for the console
  deg = ($itor(angle) / 4294967296.0) * 360.0;
  if (deg < 0) deg = deg + 360.0;

  // Randomly inject valid_in bubbles (approx 50% of the time data is invalid)
  valid_in = $random % 2;

  // Only push to our checking queue if we actually asserted valid_in
  if (valid_in) begin
    deg_queue[push_idx] = deg;
    push_idx = push_idx + 1;
  end
end
```

```
  // ========================================================
  // TEST PHASE 3: Mid-Flight Reset Assertion
  // Goal: Asynchronous/Synchronous Reset Path Coverage
  // ========================================================
  $display("Starting Phase 3: Mid-Flight Reset...");
  @(negedge clk);
  valid_in = 1;
  angle = 32'h40000000; // 90 degrees
  deg_queue[push_idx] = 90.0;
  push_idx = push_idx + 1;

  repeat(5) @(negedge clk); // Wait for data to get halfway through pipeline

  // Assert reset mid-calculation!
  rst = 1;
  valid_in = 0;
  repeat(2) @(negedge clk);
  rst = 0;

  // Note: Because we reset, the data currently in the pipeline is dead.
  // We must artificially advance our pop_idx so the monitor doesn't
  // wait for an answer that will never arrive.
  pop_idx = push_idx;

  // Flush the pipeline
  valid_in = 0;
  repeat(30) @(posedge clk);

  $display("============================================================");
  $display(" Simulation and Coverage Run Complete.");
  $display("============================================================");
  $finish;
 end

 // ------------------------------------------------------------
 // Output Monitor Block (Checking the Results)
 // ------------------------------------------------------------
 always @(posedge clk) begin
  if (valid_out && !rst) begin

    deg = deg_queue[pop_idx];
    pop_idx = pop_idx + 1;

    exp_cos = $cos(deg * PI / 180.0);
    exp_sin = $sin(deg * PI / 180.0);

    got_cos = $itor(cosine) / SCALE;
    got_sin = $itor(sine)   / SCALE;

    err_cos = rabs(got_cos - exp_cos);
    err_sin = rabs(got_sin - exp_sin);

    // Only print if there is a massive error, otherwise console gets flooded
    if (err_cos > 0.005 || err_sin > 0.005) begin
      $display("ERROR at Angle %3.1f° | Cos Err: %.4f | Sin Err: %.4f", deg, err_cos, err_sin);
    end
  end
 end

endmodule
```

3.Waveform(SimVison)

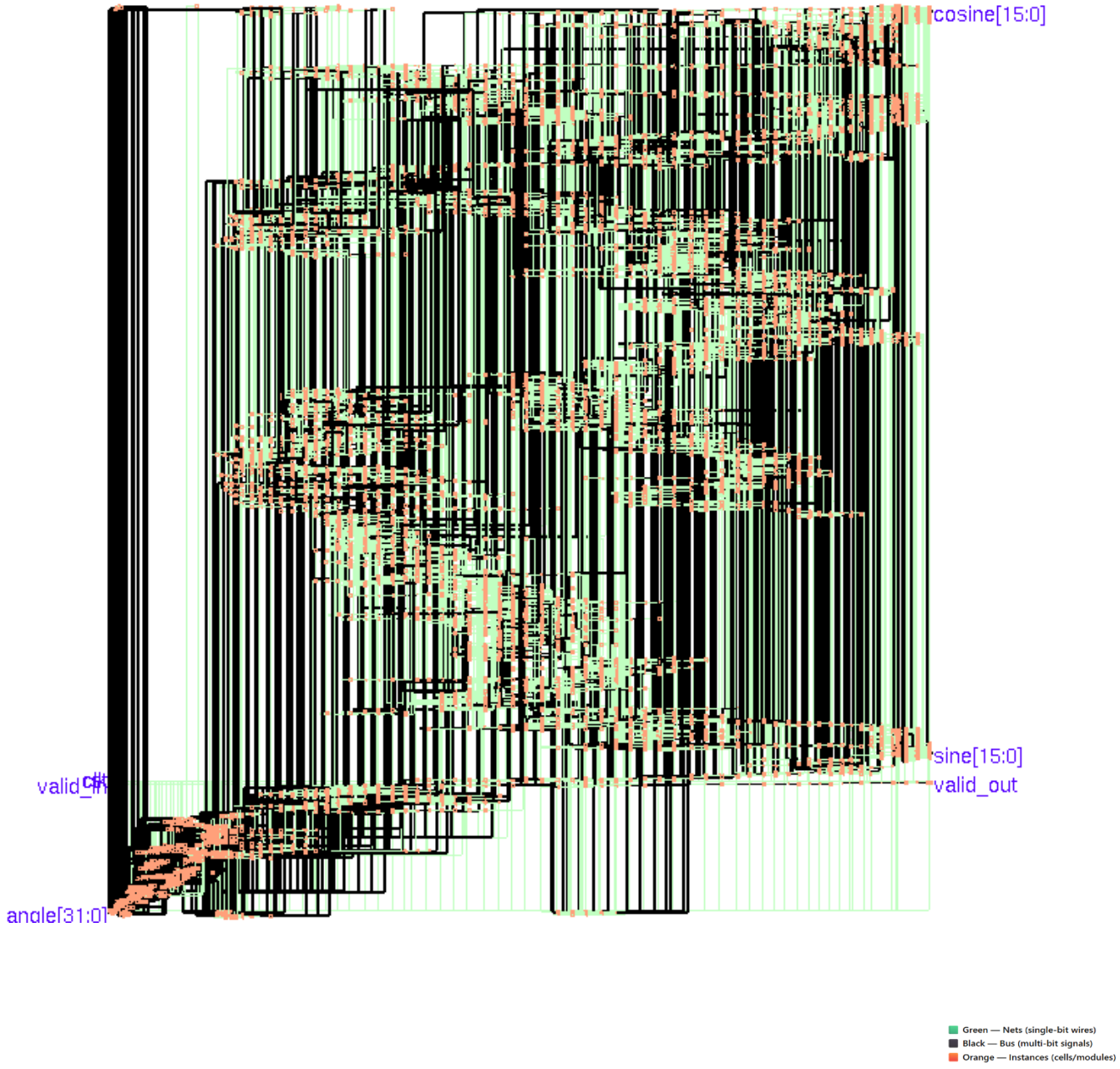


4.Coverage Analysis (Ncsim)



5.GENUS

5a) Netlist



Netlist for Cordic Algorithm

5b) Timing Report:

```
====================================================================
  Generated by:           Genus(TM) Synthesis Solution 20.11-s111_1
  Generated on:           Apr 26 2026  12:32:59 am
  Module:                 CORDIC
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
====================================================================


Path 1: MET (4 ps) Setup Check with Pin z_reg[9][30]/CK->D
          Group: clk
     Startpoint: (R) z_reg[8][0]/CK
          Clock: (R) clk
       Endpoint: (R) z_reg[9][30]/D
          Clock: (R) clk

                    Capture       Launch
        Clock Edge:+    5000            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=    5000            0

             Setup:-     217
       Uncertainty:-      50
     Required Time:=    4733
      Launch Clock:-       0
         Data Path:-    4729
             Slack:=       4

#-------------------------------------------------------------------------------------------------------
#     Timing Point      Flags   Arc   Edge   Cell     Fanout Load Trans Delay Arrival Instance
#                                                          (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------------
  z_reg[8][0]/CK        -       -     R     (arrival)    944    -    50     0       0    (-,-)
  z_reg[8][0]/Q         -       CK->Q R     DFFQX1         5  8.3    90   302     302    (-,-)
  add_135_37_I9_g1415/Y -       B->Y  F     NOR2XL         3  6.3    98    91     393    (-,-)
  add_135_37_I9_g1413/Y -       B->Y  R     NOR2XL         3  6.1   204   176     569    (-,-)
  add_135_37_I9_g1410/Y -       B->Y  F     NAND2XL        3  6.1   187   171     740    (-,-)
  add_135_37_I9_g1407/Y -       B->Y  R     NAND2BX1       3  5.0    90    97     837    (-,-)
  add_135_37_I9_g1405/Y -       B->Y  F     NAND2XL        3  6.1   168   144     980    (-,-)
  add_135_37_I9_g1402/Y -       B->Y  R     NAND2BX1       3  5.0    85    94    1074    (-,-)
  add_135_37_I9_g1400/Y -       B->Y  F     NAND2XL        3  5.0   142   126    1200    (-,-)
  add_135_37_I9_g1398/Y -       B->Y  R     NOR2XL         3  5.0   176   167    1367    (-,-)
  add_135_37_I9_g1396/Y -       B->Y  F     NOR2XL         3  6.2   114   111    1478    (-,-)
  add_135_37_I9_g1393/Y -       B->Y  R     NAND2BX1       3  5.0    70    80    1558    (-,-)
  add_135_37_I9_g1391/Y -       B->Y  R     OR2X1          3  5.0    60   111    1669    (-,-)
  add_135_37_I9_g1389/Y -       B->Y  R     OR2X1          3  5.0    60   108    1777    (-,-)
  add_135_37_I9_g1387/Y -       B->Y  F     NOR2XL         3  6.3    98    82    1859    (-,-)
  add_135_37_I9_g1385/Y -       B->Y  R     NOR2XL         3  6.1   204   176    2035    (-,-)
  add_135_37_I9_g1382/Y -       B->Y  F     NOR2XL         3  6.3   123   116    2151    (-,-)
  add_135_37_I9_g1380/Y -       B->Y  R     NOR2XL         3  6.1   205   183    2335    (-,-)
  add_135_37_I9_g1378/Y -       B->Y  F     NAND2XL        3  5.0   164   154    2488    (-,-)
  add_135_37_I9_g1376/Y -       B->Y  R     NOR2XL         3  5.0   179   174    2662    (-,-)
  add_135_37_I9_g1374/Y -       B->Y  F     NOR2XL         3  5.1   104   101    2763    (-,-)
  add_135_37_I9_g1372/Y -       B->Y  R     NOR2XL         3  5.0   173   156    2919    (-,-)
  add_135_37_I9_g1370/Y -       B->Y  F     NOR2XL         3  6.3   114   111    3030    (-,-)
  add_135_37_I9_g1368/Y -       B->Y  R     NOR2XL         3  6.1   205   181    3211    (-,-)
  add_135_37_I9_g1366/Y -       B->Y  F     NAND2XL        3  6.2   189   173    3384    (-,-)
  add_135_37_I9_g1364/Y -       B->Y  R     NOR2XL         3  6.1   213   202    3586    (-,-)
  add_135_37_I9_g1362/Y -       B->Y  F     NAND2XL        3  6.2   191   175    3761    (-,-)
  add_135_37_I9_g1360/Y -       B->Y  R     NOR2XL         3  6.1   213   203    3964    (-,-)
  add_135_37_I9_g1358/Y -       B->Y  F     NAND2XL        3  5.0   166   156    4120    (-,-)
  add_135_37_I9_g1356/Y -       B->Y  R     NOR2XL         3  5.1   182   176    4296    (-,-)
  add_135_37_I9_g1354/Y -       B->Y  R     AND2X1         3  6.1    82   185    4481    (-,-)
  add_135_37_I9_g1351/Y -       A1->Y R     OA21XL         1  1.7    53   126    4607    (-,-)
  g18634__8246/Y        -       B1->Y R     AO22XL         1  1.6    80   122    4729    (-,-)
  z_reg[9][30]/D        <<<     -     R     DFFQX1         1    -     -     0    4729    (-,-)
#-------------------------------------------------------------------------------------------------------
```

5c) Gates Report:

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 20.11-s111_1
  Generated on:           Apr 26 2026  12:33:01 am
  Module:                 CORDIC
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================


   Gate    Instances    Area    Library
-----------------------------------------
ADDFX1           174   3424.215    slow
ADDFXL            10    196.794    slow
ADDHX1             1     12.110    slow
ADDHXL             4     48.442    slow
AND2X1            47    213.446    slow
AND2XL           236   1071.770    slow
AND3X1             2     12.110    slow
AO21X1           130    885.573    slow
AO22XL           358   2709.702    slow
AOI211X1           4     21.193    slow
AOI211XL           1      5.298    slow
AOI21X1          123    558.592    slow
AOI21XL           33    149.866    slow
AOI22X1            1      6.055    slow
AOI2BB1X1         41    248.263    slow
AOI2BB1XL          8     48.442    slow
AOI31X1            3     18.166    slow
AOI32X1            1      6.812    slow
CLKINVX1         124    281.567    slow
CLKINVX4           1      6.055    slow
CLKXOR2X1         19    158.192    slow
DFFQX1           891  14162.356    slow
DFFTRXL           41    837.888    slow
DFFX1             12    227.070    slow
INVX1             73    165.761    slow
INVXL            163    370.124    slow
MX2X1             48    326.981    slow
MXI2X1             1      6.812    slow
MXI2X2             2     19.679    slow
MXI2XL           320   1937.664    slow
NAND2BX1         244   1108.102    slow
NAND2BXL          21     95.369    slow
NAND2X1            1      3.784    slow
NAND2XL          690   2089.044    slow
NAND3BX1          12     72.662    slow
NAND3BXL           1      6.055    slow
NAND3X1            8     36.331    slow
NAND3XL            3     13.624    slow
NAND4BXL           1      6.812    slow
NAND4XL            7     37.088    slow
NOR2BX1          299   1357.879    slow
NOR2BXL            3     13.624    slow
NOR2X1            11     41.630    slow
NOR2XL           415   1256.454    slow
NOR3X1             5     22.707    slow
NOR3XL             1      4.541    slow
NOR4BX1            1      6.812    slow
NOR4X1             3     18.166    slow
OA21X1             3     20.436    slow
OA21XL           168   1144.433    slow
OAI211X1          28    148.352    slow
OAI211XL           3     15.895    slow
OAI21X1          343   1557.700    slow
OAI21XL           83    376.936    slow
OAI221X1          11     83.259    slow
OAI222XL           1      8.326    slow
OAI22X1            2     12.110    slow
OAI2BB1X1        132    699.376    slow
OAI2BB1XL        203   1075.555    slow
OAI31X1            6     36.331    slow
OAI32X1            1      6.812    slow
OR2X1             88    399.643    slow
OR2XL              2      9.083    slow
OR3X1              1      6.055    slow
OR3XL              2     12.110    slow
OR4X1              1      6.812    slow
XNOR2X1            4     33.304    slow
XNOR2XL            4     33.304    slow
XOR2XL            12     99.911    slow
-----------------------------------------
total           5696  40113.429
```

Type	Instances	Area	Area %
sequential	944	15227.314	38.0
inverter	361	823.507	2.1
logic	4391	24062.608	60.0
physical_cells	0	0.000	0.0
total	5696	40113.429	100.0

5 d) Power Report:

Instance: /CORDIC
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	9.46607e-05	3.83644e-03	2.75721e-04	4.20682e-03	55.32%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.14373e-04	2.17155e-03	8.51995e-04	3.13792e-03	41.26%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	2.59654e-04	2.59654e-04	3.41%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	2.09034e-04	6.00799e-03	1.38737e-03	7.60440e-03	99.99%
Percentage	2.75%	79.01%	18.24%	100.00%	100.00%

5 e) Area Report:

```
╞=============================================================
  Generated by:           Genus(TM) Synthesis Solution 20.11-s111_1
  Generated on:           Apr 26 2026  12:33:00 am
  Module:                 CORDIC
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
=============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------
CORDIC                5696  40113.429     0.000    40113.429 <none> (D)
  (D) = wireload is default in technology library
```

5 f) Generated Netlist Report:

```
// Generated by Cadence Genus(TM) Synthesis Solution 20.11-s111_1
// Generated on: Apr 26 2026 00:32:56 IST (Apr 25 2026 19:02:56 UTC)

// Verification Directory fv/CORDIC

module CORDIC(clk, rst, valid_in, angle, cosine, sine, valid_out);
  input clk, rst, valid_in;
  input [31:0] angle;
  output [15:0] cosine, sine;
  output valid_out;
  wire clk, rst, valid_in;
  wire [31:0] angle;
  wire [15:0] cosine, sine;
  wire valid_out;
  wire [16:0] \y[14] ;
  wire [31:0] \z[14] ;
  wire [16:0] \x[14] ;
  wire [15:0] valid_pipe;
  wire [16:0] \x[0] ;
  wire [16:0] \y[0] ;
  wire [31:0] \z[0] ;
  wire [31:0] \z[3] ;
  wire [31:0] \z[6] ;
  wire [31:0] \z[9] ;
  wire [31:0] \z[10] ;
  wire [31:0] \z[11] ;
  wire [31:0] \z[12] ;
  wire [31:0] \z[13] ;
  wire [16:0] \y[3] ;
  wire [16:0] \y[13] ;
  wire [16:0] \y[12] ;
  wire [16:0] \y[11] ;
  wire [31:0] \z[1] ;
  wire [16:0] \y[1] ;
  wire [16:0] \y[10] ;
  wire [16:0] \y[9] ;
  wire [31:0] \z[8] ;
  wire [16:0] \y[8] ;
  wire [31:0] \z[7] ;
```

```
wire [16:0] \x[13] ;
wire ADD_TC_CI_OP5_n_321, ADD_TC_CI_OP5_n_324, ADD_TC_CI_OP5_n_327,
     ADD_TC_CI_OP5_n_329, ADD_TC_CI_OP5_n_331, ADD_TC_CI_OP5_n_334,
     ADD_TC_CI_OP5_n_337, ADD_TC_CI_OP5_n_339;
wire ADD_TC_CI_OP5_n_342, ADD_TC_CI_OP5_n_345, ADD_TC_CI_OP5_n_347,
     ADD_TC_CI_OP5_n_350, ADD_TC_CI_OP5_n_353, ADD_TC_CI_OP5_n_355,
     ADD_TC_CI_OP5_n_358, ADD_TC_CI_OP5_n_361;
wire ADD_TC_CI_OP5_n_363, ADD_TC_CI_OP5_n_366, ADD_TC_CI_OP5_n_369,
     ADD_TC_CI_OP5_n_371, ADD_TC_CI_OP5_n_374, ADD_TC_CI_OP5_n_377,
     ADD_TC_CI_OP5_n_379, ADD_TC_CI_OP5_n_382;
wire ADD_TC_CI_OP5_n_385, ADD_TC_CI_OP5_n_387, ADD_TC_CI_OP5_n_390,
     ADD_TC_CI_OP5_n_393, ADD_TC_CI_OP5_n_395, ADD_TC_CI_OP5_n_398,
     ADD_TC_CI_OP5_n_401, ADD_TC_CI_OP5_n_403;
wire ADD_TC_CI_OP5_n_406, ADD_TC_CI_OP5_n_409, ADD_TC_CI_OP5_n_411,
     ADD_TC_CI_OP5_n_414, ADD_TC_CI_OP5_n_431, ADD_TC_CI_OP5_n_445,
     ADD_TC_CI_OP5_n_446, ADD_TC_CI_OP5_n_447;
wire ADD_TC_CI_OP5_n_448, ADD_TC_CI_OP5_n_449, ADD_TC_CI_OP5_n_450,
     ADD_TC_CI_OP5_n_451, ADD_TC_CI_OP5_n_453, ADD_TC_CI_OP5_n_454,
     ADD_TC_CI_OP5_n_455, ADD_TC_CI_OP5_n_456;
wire ADD_TC_CI_OP5_n_457, ADD_TC_CI_OP5_n_459, ADD_TC_CI_OP5_n_461,
     ADD_TC_CI_OP5_n_464, ADD_TC_CI_OP5_n_485, ADD_TC_CI_OP21_n_300,
     ADD_TC_CI_OP21_n_303, ADD_TC_CI_OP21_n_306;
wire ADD_TC_CI_OP21_n_309, ADD_TC_CI_OP21_n_311,
     ADD_TC_CI_OP21_n_312, ADD_TC_CI_OP21_n_314,
     ADD_TC_CI_OP21_n_317, ADD_TC_CI_OP21_n_320,
     ADD_TC_CI_OP21_n_322, ADD_TC_CI_OP21_n_325;
wire ADD_TC_CI_OP21_n_328, ADD_TC_CI_OP21_n_330,
     ADD_TC_CI_OP21_n_333, ADD_TC_CI_OP21_n_336,
     ADD_TC_CI_OP21_n_338, ADD_TC_CI_OP21_n_341,
     ADD_TC_CI_OP21_n_344, ADD_TC_CI_OP21_n_346;
wire ADD_TC_CI_OP21_n_349, ADD_TC_CI_OP21_n_352,
     ADD_TC_CI_OP21_n_354, ADD_TC_CI_OP21_n_357,
     ADD_TC_CI_OP21_n_360, ADD_TC_CI_OP21_n_362,
     ADD_TC_CI_OP21_n_365, ADD_TC_CI_OP21_n_368;
wire ADD_TC_CI_OP21_n_370, ADD_TC_CI_OP21_n_373,
     ADD_TC_CI_OP21_n_376, ADD_TC_CI_OP21_n_378,
     ADD_TC_CI_OP21_n_381, ADD_TC_CI_OP21_n_390,
     ADD_TC_CI_OP21_n_401, ADD_TC_CI_OP21_n_402;
wire ADD_TC_CI_OP21_n_403, ADD_TC_CI_OP21_n_404,
     ADD_TC_CI_OP21_n_405, ADD_TC_CI_OP21_n_407,
     ADD_TC_CI_OP21_n_408, ADD_TC_CI_OP21_n_409
```

6.)INNOVUS

6 a)

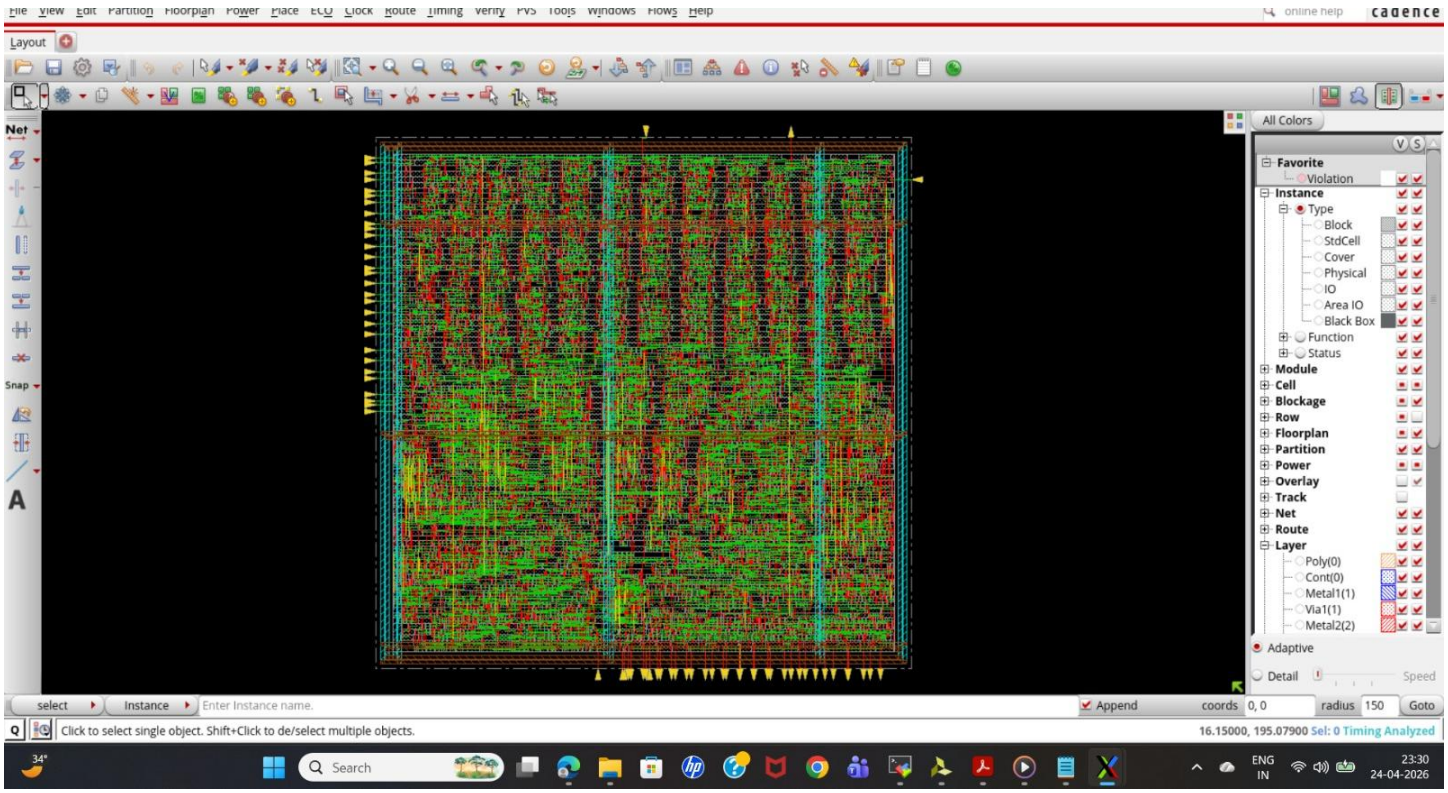
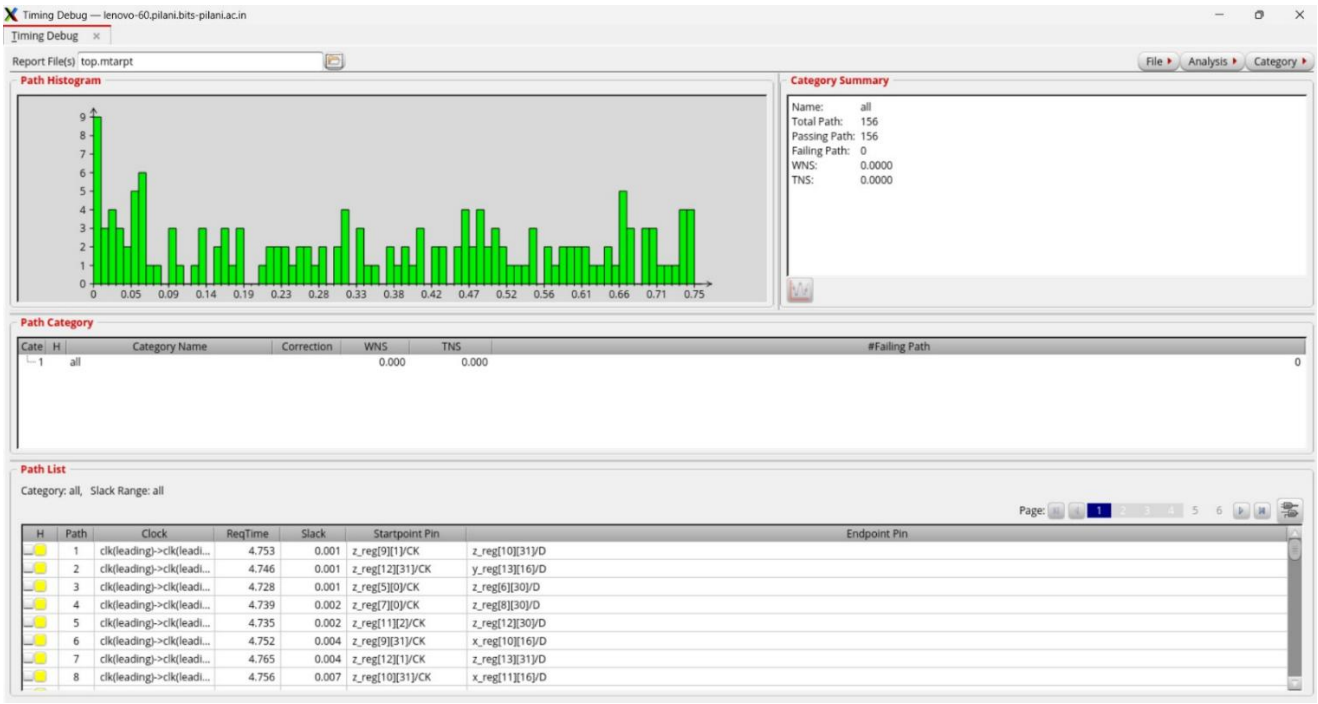


Fig: Layout

6 b) Timing Report:



No setup Violation in any path

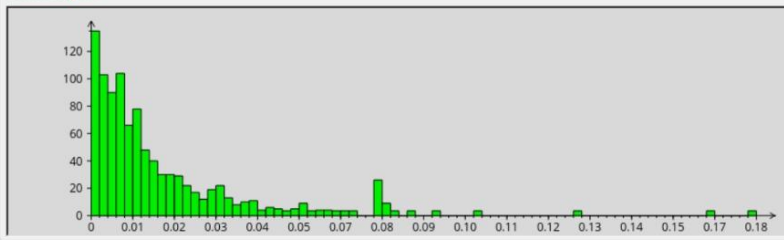
Timing Debug — lenovo-60.pilani.bits-pilani.ac.in

Timing Debug

Report File(s) top.mtarpt

File Analysis Category

Path Histogram



Category Summary

Name:	all
Total Path:	977
Passing Path:	977
Failing Path:	0
WNS:	0.0000
TNS:	0.0000

Path Category

Cate	H	Category Name	Correction	WNS	TNS	#Failing Path
1		all		0.000	0.000	0

Path List

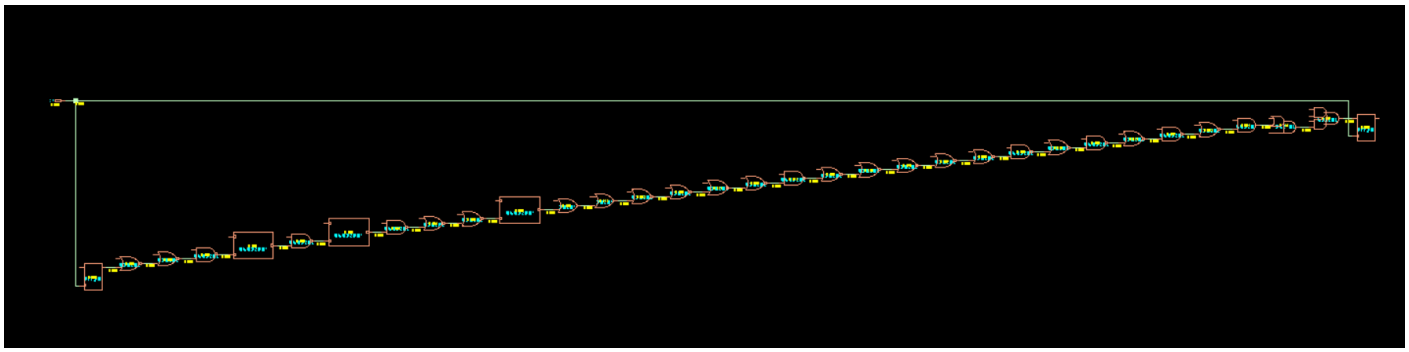
Category: all, Slack Range: all

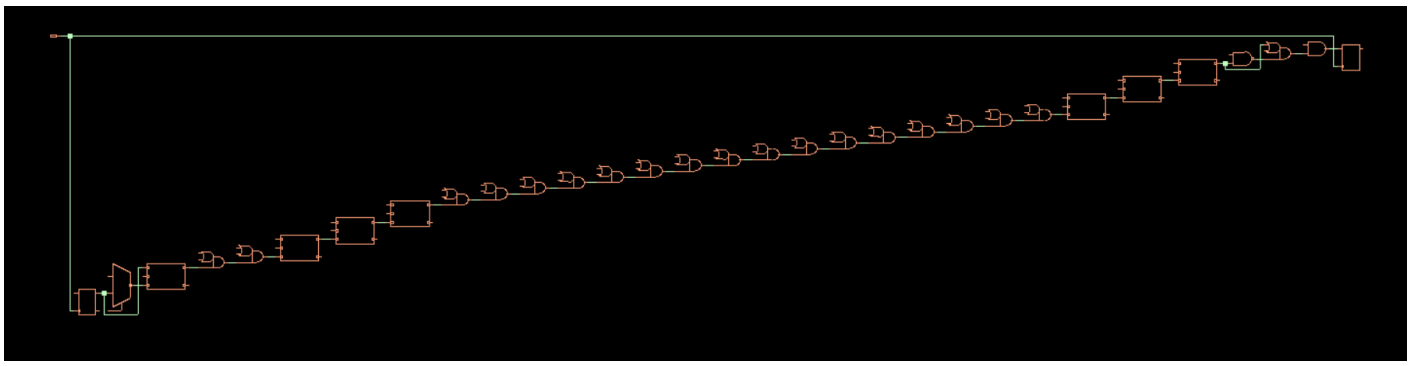
Page: 1

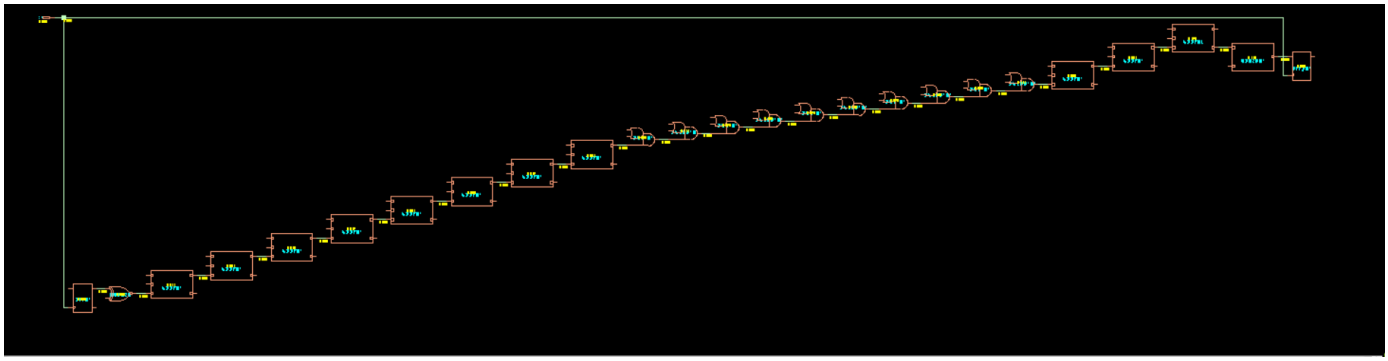
H	Path	Clock	ReqTime	Slack	Startpoint Pin	Endpoint Pin
	1	clk(leading)->clk(leadi...	0.178	-0.000	z_reg[11][2]/CK	z_reg[12][2]/D
	2	clk(leading)->clk(leadi...	0.173	0.000	z_reg[2][8]/CK	z_reg[3][8]/D
	3	clk(leading)->clk(leadi...	0.173	0.000	z_reg[1][16]/CK	z_reg[2][16]/D
	4	clk(leading)->clk(leadi...	0.172	0.000	z_reg[8][8]/CK	z_reg[9][8]/D
	5	clk(leading)->clk(leadi...	0.169	0.000	z_reg[11][0]/CK	z_reg[12][0]/D
	6	clk(leading)->clk(leadi...	0.173	0.000	z_reg[9][15]/CK	z_reg[10][15]/D
	7	clk(leading)->clk(leadi...	0.174	0.000	z_reg[4][15]/CK	z_reg[5][15]/D
	8	clk(leading)->clk(leadi...	0.180	0.000	x_reg[3][2]/CK	x_reg[4][2]/D

No hold time violation in any Path

6 c) Some worst path schematic







6 d) DRC Report

```
  VERIFY DRC ..... Starting Verification
  VERIFY DRC ..... Initializing
  VERIFY DRC ..... Deleting Existing Violations
  VERIFY DRC ..... Creating Sub-Areas
  VERIFY DRC ..... Using new threading
  VERIFY DRC ..... Sub-Area: {0.000 0.000 122.880 120.960} 1 of 4
  VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.
  VERIFY DRC ..... Sub-Area: {122.880 0.000 242.440 120.960} 2 of 4
  VERIFY DRC ..... Sub-Area : 2 complete 0 Viols.
  VERIFY DRC ..... Sub-Area: {0.000 120.960 122.880 238.090} 3 of 4
  VERIFY DRC ..... Sub-Area : 3 complete 0 Viols.
  VERIFY DRC ..... Sub-Area: {122.880 120.960 242.440 238.090} 4 of 4
  VERIFY DRC ..... Sub-Area : 4 complete 0 Viols.

  Verification Complete : 0 Viols.
```

6 e) Power Report

```
Total Power
-----------------------------------------------------------------------------------------
Total Internal Power:        8.25425018            75.5195%
Total Switching Power:       2.46667450            22.5680%
Total Leakage Power:         0.20903366             1.9125%
Total Power:                10.92995832
-----------------------------------------------------------------------------------------


Group                            Internal    Switching     Leakage       Total  Percentage
                                 Power       Power         Power         Power  (%)
-----------------------------------------------------------------------------------------
Sequential                          4.473      0.5854      0.09466         5.153       47.15
Macro                                   0           0            0             0           0
IO                                      0           0            0             0           0
Combinational                       3.781       1.881       0.1144         5.777       52.85
Clock (Combinational)                   0           0            0             0           0
Clock (Sequential)                      0           0            0             0           0
-----------------------------------------------------------------------------------------
Total                               8.254       2.467        0.209         10.93         100
-----------------------------------------------------------------------------------------


Rail                   Voltage   Internal    Switching     Leakage       Total  Percentage
                                 Power       Power         Power         Power  (%)
-----------------------------------------------------------------------------------------
Default                    0.9      8.254       2.467        0.209         10.93         100


-----------------------------------------------------------------------------------------
*       Power Distribution Summary:
*              Highest Average Power:              z_reg[9][31] (DFFX1):           0.0157
*              Highest Leakage Power:              z_reg[1][29] (DFFX1):        0.0001344
*                Total Cap:      3.33881e-11 F
*                Total instances in design:  5696
*                Total instances in design with no power:     0
*                Total instances in design with no activty:     0

*                Total Fillers and Decap:     0
-----------------------------------------------------------------------------------------
```